

Paper Review Report

Benchmarking Open Source Distributed Property-Graph Databases with Portable Mixed Workloads under Elastic Scale-Out

Mustafa Can Çalışkan, 504251544

BLG 564E – Advanced Database Systems

Istanbul Technical University

April 2026

Introduction

This report includes the literature review of my term project that I will do in the scope of BLG564E Advanced Database Systems course. Current literature, especially on SNB BI [1], FinBench [2] and SNB Interactive v2 [3] datasets, offers strong benchmark studies for OLTP and OLAP. In the same way, there are also recent papers in the literature such as Galaxybase [4], G-Tran [5], ByteGraph [6] and BACH [7], which explain the importance of mixed graph workloads. However, the thing that I identified as missing is that there is still no standard, portable and multi-system benchmark study for open source distributed property-graph databases, which combines publicly available graph workload components, clearly states the compromises coming from portability, and evaluates them under a controlled elasticity event with a consistent metric set. For this reason, the direction of the study I plan is as follows: the systems to be benchmarked are determined as NebulaGraph [8], JanusGraph [9], HugeGraph [10], Dgraph [11] and ArangoDB [12]. The publicly available datasets that will shape the benchmark are LDBC SNB BI [1], LDBC SNB Interactive v2 [3] and LDBC FinBench [2]. During the evaluation, instead of trying to show that one single system is universally the best for all usage scenarios, I aim to make more realistic observations and find what kind of balance these systems have between throughput, latency and inference when a portable workload mix is executed, and to observe the changes when scale-out events happen while the workload is still running. I think that these observations will make my study different from the existing studies in the current literature.

Part 1: Review of Related Papers

1. The LDBC Social Network Benchmark: Business Intelligence Workload [1]

SNB BI is one of the strongest benchmark definition paper on the analytical side of this topic. This paper talks about the lack of a realistic, portable and graph aware OLAP benchmark for systems that need to evaluate things like pattern matching, graph aware join, path finding, temporal changes and skewed data distributions. I think the strongest side of this paper is its methodology. SNB BI uses LDBC's choke point-driven design, a scalable temporal data generator, batch and trickle updates, and also a parameter-curation pipeline to keep runtimes more stable and understandable [1]. It also defines two benchmark metrics, which are power and sustained throughput, and clearly separates the benchmark specification from the example results. This is important because it shows that a serious benchmark study can be both experimentally concrete and also auditable [1].

The biggest contribution of SNB BI is not choosing one winner system. More importantly, it presents a high-quality graph OLAP benchmark and shows its portability on both a relational implementation and a graph DBMS implementation [1]. Because of this, it is also a very important study for the project I am planning. In my planned study, reproducing the full benchmark suite

of SNB BI on all five systems would be very hard and too ambitious. But still, SNB BI gives me the most useful design points for the analytical side of the benchmark. These are clear benchmark boundaries, stable parameter generation, a clear workflow, and careful interpretation of the results [1].

But SNB BI also has some limitations, and these are important for defining the scope of my study. If we look at it alone, SNB BI is not a mixed transactional and analytical benchmark. It includes updates and a concurrency-aware point of view, but still, in the end, it is not an OLTP+OLAP benchmark, it is more an OLAP workload [1]. Also, because its queries are prepared in a more sophisticated way on purpose, it can increase portability problems between systems that have different query languages or different feature support. Because of this, in the benchmark I plan, I am thinking to use SNB BI as a design source and as the main OLAP workload family. But most probably, instead of trying to provide full benchmark compliance on every system, I will use a more portable subset of the analytical queries.

2. The LDBC Financial Benchmark: Transaction Workload [2]

FinBench is the most recent benchmark definition study among the uploaded papers, and it is especially very important because it carries graph benchmarks beyond the social-network domain. The problem this paper deals with is that older graph benchmarks do not fully reflect the important properties of financial workloads. These include things like edge multiplicity, temporal skew, hub vertices, read-write transactional patterns, time window filtering and recursive path filtering [2]. Its methodology also follows the LDBC tradition. So it uses a scalable data generator, update streams, factor tables, parameter generation for stable runtimes, and a benchmark driver that directs the query mix according to time windows [2].

The strongest contribution of FinBench is that it shows domain-specific choke points in a more clear way. The paper shows transactional graph benchmarking should not be reduced only to simple 1-hop reads or social-network style traversals [2]. It also includes read-write queries whose execution path can change depending on previous read results. This becomes a very relevant point for studies that examine concurrent interference and transactional stress [2]. The experimental part is also kept more measured in one sense on purpose. So it does not present itself like a study proving that one system always wins. Rather, it shows that this benchmark can reveal new bottlenecks on the graph language, storage and execution side [2]. I think this is also the right approach for a benchmarking paper.

From the perspective of the project I am planning, its main limitation is that this is still not a full hybrid graph benchmark, but more a transaction workload. The paper itself already says this clearly in the future work part, and leaves an OLAP workload for financial scenarios and broader HTAP benchmarking directions to future studies [2]. This future-work part is very useful for my project, because it confirms that the hybrid-graph benchmark gap is still not closed, it is still open. Because

of this, in my planned project I will not present FinBench as a complete solution. Rather, I will use it as one of the publicly available workload families from which portable transactional and read write patterns can be derived.

3. The LDBC Social Network Benchmark Interactive Workload v2 [3]

SNB Interactive v2 is one of the main benchmark definition papers for the project I am planning. This paper revisits the old SNB Interactive workload and says that today's transactional graph benchmarks need deeper delete semantics, stronger parameter curation, bigger scale factors, a clearer workload scheduling structure, and more clear auditing rules. I think its methodology is one of the most useful sides of this paper. Because it discusses topics like relevance, portability, scalability and simplicity in a more organized way. On the workload side, it also allows multiple implementation styles, puts rules for invalid runs, collects detailed latency statistics, and asks for stronger transactional guarantees compared to older versions, because delete operations can create consistency problems when isolation is weak [3].

I think the strongest side of this paper is not only giving a high throughput result, but offering a more mature structure in terms of benchmark engineering. The paper explains why on-the-fly workload generation is not enough for transactional graph workloads that have deep dependency chains, why stable runtimes are needed, and why scheduling, temporal bucketing and executability rules should be defined clearly on the benchmark driver side. It also brings a 95 percent on-time requirement, and discusses topics like full disclosure reports and ACID testing [3]. I think these are also strong points that make a benchmark move beyond being just something that works, and make it more scientifically reliable.

Its main limitation is actually similar to FinBench. Interactive v2 is still an OLTP benchmark. Even though it is more advanced compared to older transactional graph workloads, by itself it does not offer a mixed protocol that evaluates transactional and analytical sides together [3]. From the perspective of the project I am planning, I do not think this is a weakness that should be ignored, but rather a boundary that should be used correctly. Because of that, I am thinking to use Interactive v2 as one of the main public sources for portable transactional operations, update scheduling ideas, and latency discipline. But in the final benchmark, instead of claiming full Interactive v2 compliance for all systems, it looks more reasonable to take and use the more feasible and portable transactional graph patterns and scheduling principles from it.

4. Galaxybase: A High Performance Native Distributed Graph Database for HTAP [4]

Galaxybase is one of the system papers that is closest to the problem that I want to study. This paper examines how a distributed graph database system can support both transactional and analytical operations on large graphs in the same structure. In other words, it looks at how small and fast

operations on one side, and larger analysis-oriented queries on the other side, can run in the same system. On the storage side, it also proposes some special structures of its own. For example, what it calls Log-Structured Adjacency List can be thought of as a structure that keeps graph connections in a more organized way and in a way that is more suitable for write operations. Edge Pages are also used to store edge information, meaning the links in the graph, in a more efficient way. Besides this, it brings together different mechanisms such as 2PC, Raft, 2PL and MVCC [4]. These are, briefly, methods used in distributed systems so that machines can make the same decision, the data can stay consistent, and operations coming at the same time do not disturb each other too much. The aim of the paper is to show that OLTP and OLAP operations can work together while blocking each other less [4]. The authors both give important ideas about system architecture and try to show experimentally why HTAP-style graph processing is needed. The HTAP idea here, briefly, is combining daily transaction workload and the analytics side in the same system.

The strongest side of Galaxybase is that it can directly show why mixed graph workloads are important in real life. Because some studies only look at one workload type. For example, only the transactional side or only the analytical side is examined. But instead of separating these two, Galaxybase is designed as a system that tries to handle both at the same time. The paper also reports strong results in terms of throughput and scale-out [4]. Here, throughput means how much work the system can do in a certain time. Scale-out is related to how the performance changes when new machines are added to the system as the need increases. Because of this, this study is very important for the motivation of my project. Especially in the part of the final paper where I will explain why this topic matters, it gives me a good basis.

But I think its limitation is also very clear. Galaxybase is not a benchmark-definition paper, but more a system paper that introduces its own system. So it does not give a common benchmark rule that can be applied in the same way on different systems. It compares its own system with some baseline systems, but it does not turn this into a portable methodology that we can easily carry to open source systems with different query languages, different features and different installation structures. My term project is also not about developing a new graph HTAP engine. I am mostly aiming to compare existing open source systems under a common benchmark protocol. Because of that, Galaxybase is not a benchmark plan that I will use directly. It becomes more a valuable reference point that shows why the mixed workload idea is important and helps me understand which measurements are more meaningful.

5. G-Tran: A High Performance Distributed Graph Database with a Decentralized Architecture [5]

G-Tran is one of the most useful references for especially the concurrency and transactional side of my project. This paper looks at the main difficulties in distributed graph transaction processing. For example, there are irregular access patterns, which means system does not always reach the data

in same way. There are also large read/write sets, meaning sometimes one operation has to read and write too much data at the same time. Also high abort rates are there, which means because of conflicting operations, cancellations become more. And centralized coordination bottlenecks also exist, meaning central structures in the system can slow things down. Why these problems are difficult, the paper explains clearly. On the methodology side, it uses a structure that aims to keep data in memory and support fast communication between machines, it builds an execution logic that reduces central control, and it combines an approach that uses multiple versions of data to manage concurrent operations more easily [5]. While doing this, it also supports strong transactional guarantees that help keep operations consistent. Another important point is that it supports a wide range of Gremlin steps, because this can be useful when thinking about portability across different graph query interfaces [5].

The experimental contribution of the paper is also important. It gives strong results in throughput, scale-up, and scale-out, and it compares with systems like JanusGraph, ArangoDB, Neo4j, and TigerGraph. On the LDBC-S dataset, it also shows that multi-hop queries, which require deeper and more complex traversals inside the graph, can create large latency differences between systems. This is important for my project, because it shows that workloads with deeper traversals and irregular access patterns can separate systems more clearly. Because of that, I do not think it would be right to reduce these kinds of workloads completely to simple key lookups.

But this paper also has a limitation. Mostly on the OLTP side this study stays. In other words, what happens when heavy analytical queries and transactional graph operations are running at same time, this it does not really examine. Also, scale-out behavior under a mixed workload defined at benchmark level, it does not study. Because of that, I am thinking to use G-Tran more like an evidence showing graph transaction processing is now a mature systems problem, and also like a reference helping me to choose meaningful transactional patterns and latency metrics. But for my project to be meaningful, exact copying of the structure in G-Tran is not needed.

6. ByteGraph: A High-Performance Distributed Graph Database in ByteDance [6]

ByteGraph looks like a useful industry reference for this project because it separates workload types like OLTP, OLSP, and OLAP in a more clear way and it tries to show what really happens in large scale graph serving systems. Here, OLTP mostly means short and fast transactional queries. OLSP is more about online serving, so the side where system answers instant requests and also has real-time writes. OLAP is more on analytical side, with heavier and more complex queries. The paper says workloads close to real life do not change only because of read/write ratio. Things like traversal depth, meaning how many steps a query goes inside the graph, burstiness, meaning sudden increase in load, and performance expectations like throughput, latency, error rate, and freshness also change. Here, throughput means how much work system can do in a certain time, latency means how long it takes to get answer, error rate means how often failures happen, and freshness

means how up-to-date the data stays. The paper also looks at query latency, throughput, scalability, and what happens when sudden load comes. Scalability means how system behaves when it grows, or when the load gets higher. It also says different query types can create different bottlenecks, meaning different slowdown points inside the system [6].

I think the strongest side of ByteGraph for this project is the idea behind it. It shows that graph workloads should not be treated like they are all the same. For example, on OLTP side, lighter 1-hop reads, meaning simple one-step reads inside the graph, and small write operations are more common. On OLSP side, simple reads and real-time writes are existing together. On OLAP side, there are deeper traversals, meaning queries pass through more steps in graph, and also heavier queries. This distinction feels very useful for my term project. Because instead of building everything as one huge benchmark, it helps me think about a more portable and step-by-step mixed workload structure. So instead of loading everything at once, it becomes possible to arrange workloads with different levels and difficulty in a more reasonable way.

But this also has some serious limitations. ByteGraph is not an open benchmark paper. Also, the workload descriptions it uses do not come from standard public benchmarks, they are more inspired by ByteDance’s own internal use cases [6]. Because of that, it is not a direct candidate for the open source system comparison that I will do at the end. So I think the main contribution of ByteGraph is helping me explain in the final paper why I organize the scenarios as separate transactional tests, mixed tests, and heavier analytical tests. So it is not a paper that directly decides which system I will choose, but more like a reference that gives ideas about how the experimental structure should be built.

7. BACH: Bridging Adjacency List and CSR Format Using LSM-Trees for HGTAP Workloads [7]

BACH is a very relevant paper for this study, because instead of focusing only on one query type, it focuses on storage-engine design for HGTAP-style graph workloads. So the main issue here is not only the queries, but also how the data is stored inside the system and how it is organized. The paper proposes a design combining adjacency-list style graph storage and CSR-style more efficient scanning, based on LSM-tree logic [7]. I think this is important, because one of the main problems in hybrid graph processing is this: transactional updates and analytical neighborhood scans usually want different storage layouts. In other words, a structure good for one side may not work equally well for the other side. Exactly with this tension, BACH tries to deal.

The strongest side of BACH is that hybrid behavior, more directly it can measure. The paper includes interactive, BI, and mixed tests, and it reports that BACH variants give better throughput and lower analytical latency than some competitors, especially in mixed or real-time analytical scenarios [7]. This makes BACH a strong support for my planned project, especially because I want to focus not only on throughput alone, but also on how workloads affect each other, which

means the interference side. Also, this paper supports another idea too: graph HGTAP is not only a query language problem or a benchmark driver problem, but also very much related to storage layout.

But it also has a limitation. Like Galaxybase, this is also a more system-centered paper, so it does not offer a portable benchmark protocol that can be easily applied to different systems. BACH also reflects a specific design understanding and a specific implementation environment. Because of this, it is not easy to take it and directly adapt it to all ready open source systems. From the perspective of my planned benchmark, BACH is most valuable for this reason: it shows that mixed graph workloads create meaningful trade-offs on the storage-engine side, and these need to be measured directly. But BACH is not the direct target of my benchmark, and I will not claim that I reproduced the storage innovations in this paper one by one.

8. General Evaluation of the Selected open source Systems

Besides reviewing benchmark-definition papers and methodology-oriented system papers, I also include a short evaluation of the five open source systems that I plan to benchmark: NebulaGraph [8], JanusGraph [9], HugeGraph [10], Dgraph [11], and ArangoDB [12]. I include these systems not because they are same, but because together they create a meaningful comparison space for studying portable mixed workloads on distributed graph oriented systems.

NebulaGraph. NebulaGraph [8] is a distributed graph database which is directly positioned around large scale graph storage and query execution. For this project, its main strong side is that it represents more native distributed graph engine perspective. At the same time, its own query model and deployment structure may create some portability and setup difficulties during benchmarking.

JanusGraph. JanusGraph [9] is a flexible graph database framework built on top of external storage and indexing backends. Its strong side is that it represents a highly configurable and also well-known open source graph stack. But from benchmarking side, its backend dependence and configuration choices can make deployment and fair comparison more complicated.

HugeGraph. HugeGraph [10] is another open source graph database which is presented more directly as a graph oriented system. I think its strong side is that it gives more clear graph database identity compared to more general purpose systems. But on the other hand, distributed deployment and setup side can still require extra effort depending on selected configuration. So for benchmarking it looks useful, but also it may bring setup difficulty.

Dgraph. Dgraph [11] is important because it represents a different graph system design in terms of query model and data model. This makes it valuable as a comparison point, because it can show how portability constraints affect benchmark design. At the same time, this same difference can make workload translation harder than more conventional property-graph systems.

ArangoDB. ArangoDB [12] is included because it gives distributed graph support inside a wider multimodel database design. I think its strong side is that it is a practical and well known open source system, and it also supports graph features together with distributed deployment. But in this benchmark context, its multimodel character can also become a limitation, because it is not as graph centered as some other systems in this list.

Overall, I selected these five systems because together they make a useful comparison space in the open source distributed graph database area. This is useful not only for asking which system performs better, but also for understanding how differences in system design, portability, and deployment can affect benchmark results. For me, the point here is not only speed comparison, but also seeing how these system differences reflect to the final benchmark results.

Synthesis and Research Gap

When these papers are looked at together, a quite clear picture appears. Benchmark-definition papers provide a stronger base in topics like workload design, parameter curation, portability logic, and auditability [3, 2, 1]. The studies on the system-paper side show that graph workloads are not all same, and in mixed or hybrid scenarios, meaningful differences really appear in topics like storage layout, concurrency control, and execution architecture [5, 7, 6, 4]. open source systems such as NebulaGraph, JanusGraph, HugeGraph, Dgraph, and ArangoDB also show that when doing this kind of semester project, differences in deployment model and query interface also need to be taken into account [8, 9, 10, 11, 12]. But despite all this, I think still there is not, in the literature and in one place, a practical and portable benchmark protocol for open source distributed property-graph databases, which can really be applied in one semester and evaluated under same mixed workload and scale-out scenarios.

The gap that my planned study focuses on is exactly this point. So I will not make a very broad claim like “hybrid graph processing has not been studied at all.” Instead, I plan to say something more narrow but more correct: in the recent literature, still there is a lack of a reproducible, portable, open, and multi-system benchmark procedure. Such a procedure should combine public graph OLTP and OLAP workload building blocks, should measure mixed workload interference directly, should explain clearly the compromises coming because of portability, and also should include an explicit scale-out event during execution under same evaluation protocol. For my semester project, I think this more focused approach is both more reasonable and more useful.

It also makes the benchmark more realistic. Because instead of acting like three different public

benchmark families can be merged without any loss, more reasonable it looks to move forward with a smaller, but more clear and portable subset.

Part 2: How to Write a Benchmarking Paper

In this section, I summarize what I learned from reading benchmarking papers about graph data management systems and related workloads. My aim here is not to repeat related work discussion from Part 1, but more to explain in tutorial style how I think a strong benchmarking paper should be written. After reading several papers on graph OLTP, graph OLAP, mixed workloads and scalability, I noticed good benchmarking papers usually have some similar strong sides. They define their scope very clearly, they design their experiments in a more disciplined way, and they present their results not like disconnected graphs and tables, but more like parts of one coherent scientific story. Based on these observations, below I discuss how this kind of paper should be organized, how the experiments should be designed, and how different parts should be connected with each other to produce a more consistent and convincing paper.

2.1. How a Benchmarking Paper Should Be Organized

My first observation is that a benchmarking paper needs to define its scope very early and very clearly. In the introduction, the reader needs to understand quickly what is benchmarked, why this problem matters, and which gap in the literature this paper is trying to address. I think this is even more important in graph data management, because systems can be very different from each other in terms of data model, query language, deployment model, and target workload. If the scope is not fixed clearly at the beginning, then the rest of the paper becomes harder to trust. Because of that, I think a good introduction should not only motivate the problem, but also explain clearly the benchmark setting, the type of systems studied, and the main evaluation questions.

After the introduction, I think the related work part should explain how the planned benchmark is different from earlier benchmark-definition papers and also from system papers which evaluate their own architectures. In my opinion, this part should not be written like a long summary list. Instead of that, the literature should be grouped in a more meaningful way. For example, one group can be papers that define public graph benchmarks, another group can be papers focusing on mixed or hybrid graph workloads, and another group can be papers about systems whose design is giving motivation to the benchmark. I think this kind of structure helps the reader to see more clearly where the new study stands in the literature.

One more thing that I noticed is that strong benchmarking papers usually have a separate section for explaining the benchmark design itself. I think this is one of the most important parts of the paper. In this section, things like workload model, dataset choice, metrics, system selection criteria, and

fairness assumptions should be defined clearly. If the paper is using only some subset of an existing benchmark, then this decision should also be explained here in a direct and transparent way. So in short, this section should make the reader understand not only what is measured, but also why this benchmark design is a reasonable experimental abstraction.

Another section which I think should be given clearly is the experimental setup part. A benchmarking paper becomes much more believable when it explains clearly things like hardware configuration, software versions, deployment assumptions, tuning policy, data loading choices, warm-up procedure, repetition policy, and timeout rules. In practice, many disagreements between benchmark results come not from the benchmark idea itself, but from setup differences staying hidden. Because of that, not like a small implementation detail should this section be treated. More like one of the main scientific parts of the paper, I think it should be.

I also think the results part should go in a more disciplined order. In the papers I read, the most effective result sections were usually starting with simpler baselines, and after that moving to more complex workload combinations. This way, I think, is the correct one. First, isolated workload behavior should be shown by the paper, so that the reader can understand the basic strong and weak sides of each system. Then to concurrent or mixed workloads it should move, because interference and trade-off become visible mostly there. And at the end, if there are also scalability or elasticity results in the paper, after the basic behavior is understood should these be given. More natural becomes the flow in this way, and easier to interpret become the results also.

Finally, I think a good benchmarking paper should also include the threats to validity part in a clear and open way. Especially in graph systems, there are always some issues. For example, feature mismatch can happen, meaning systems may not support exactly same features. Or there can be some compromises during query translation, dataset limitations, or unequal situations on the deployment side. I do not think writing these things clearly makes the paper weaker. The opposite, I think. Stronger makes the paper it, because it shows that the study was designed in a careful and honest way.

2.2. How Experiments Should Be Designed

The main lesson that I got from reading these papers was this, benchmarking experiments should be designed around comparability and interpretability. so it is not enough to just run many queries on many systems. What is needed is, the experiment design should show that systems are compared on a common and meaningful ground.

The first design principle is defining a common and executable subset for me. Because in graph systems not every database supports same query language, same optimizer behavior, same indexing model or same traversal logic. Because of this, a benchmarking paper needs to choose a workload subset that can be applied fairly across systems. If one query or one feature cannot be moved

to another system without changing its meaning too much, this should be said openly. I think this is much better than acting like everything is fully comparable while actually there are serious differences between implementations.

The second principle is separating workload classes carefully. A strong benchmark should show clearly the difference between transactional behavior, analytical behavior, and mixed behavior. If these are reduced into one average score too early, then the reader cannot understand why one system behaves in that way. For me, isolated baselines are very important here. I mean, first the paper should show how each system behaves under only transactional and only analytical conditions. After that, it should move to mixed workload results. In this way, easier becomes interpreting the interference effects.

The third principle is fairness on the system setup side. A benchmarking paper should explain clearly how indexes are chosen, how cache effects are handled, how data is loaded, and under which limits tuning is done. I think fairness does not mean forcing same settings on systems which are already different by nature. More like, it means giving each system a fair chance for performing well under a documented and balanced tuning policy. When the reader sees that authors did not make random optimizations for only selected systems, but made more careful and consistent decisions, much more convincing the paper becomes.

Another principle which I think very important is progressive experiment design. Good papers usually start with simple validation experiments, then move to performance baselines, after that to mixed workloads, and in the end to things like scalability or elasticity. This order, I think, is also good from teaching side, because it helps reader build intuition step by step. If from beginning paper jumps directly into complex mixed workload figures, without showing first baseline behavior, then much harder becomes understanding the results.

One more thing that I noticed is this: benchmarking experiments should not be designed with idea of “let us produce as many figures as possible,” but more with idea of “which concrete question are we answering.” For example, one experiment can show how well system handles transactional throughput. Another one can show how much analytical latency increases under concurrency. Another experiment can show how fast performance becomes stable after a scale-out event. So each experiment should have a clear purpose. This, I think, makes the final paper more readable, and also reduces the risk of collecting unnecessary results which are difficult to interpret.

2.3. How to Build a Consistent and Interesting Scientific Story

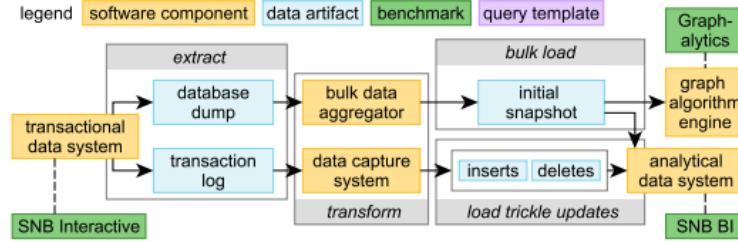
What I understood from the papers I reviewed is this: the most convincing benchmarking papers are not always the ones doing the highest number of experiments. In my opinion, the really strong ones are the papers which can turn their experiments into one clear and consistent scientific story, instead of just giving them one by one. For me, this story should start with one clear evaluation

question, and after that it should continue with result sections which say more and more thing step by step. A good scientific story should first make the scope of the benchmark and the expectation of the reader clear. I mean, paper should say in beginning which class of systems is studied, what kind of workloads are important, and what kind of claim it will make. I think when writing benchmarking paper, making too big claims is not good thing. A question with clearer limits becomes more convincing than very broad claim, I think. For example, instead of saying one system is best everywhere, asking how some systems make a balance between throughput, latency, and interference under portable mixed workload sounds more reasonable to me. Also results part should not go randomly I think. Each subsection should answer one question and should prepare the reader for next one. This I saw many times in strong papers. First baseline is established, then more difficult situations like stress, concurrency, scale, or elasticity are added. After that, why the behavior changes is explained. This kind of flow, I think, makes the paper feel not only descriptive, but more scientific. Another thing that I found useful is this: good benchmarking papers keep connecting design decisions to the results. If one system behaves differently under mixed workload, then this should be connected to things like architecture, query model, storage layout, or concurrency control side. For me, graphs and tables become really meaningful only when they are given together with these kind of system-level explanations. Also I think a benchmarking paper should not give every result with same weight. Not every finding is equally important. The most critical results should stay in the main flow of the paper, and smaller supporting results can be given in a shorter way. When it is like this, the paper stays more focused, and the reader also does not get lost between too many unnecessary measurements. Finally, I think when the paper finishes, it should return again to the research question from the beginning. The conclusion part should not only repeat the results of tables. Instead of this, it should explain what the benchmark taught us about the systems, what kind of trade-offs it showed, and which limitations are still remaining. When it is like that, for me, the paper feels more complete, and the real contribution of the benchmark also becomes more clear.

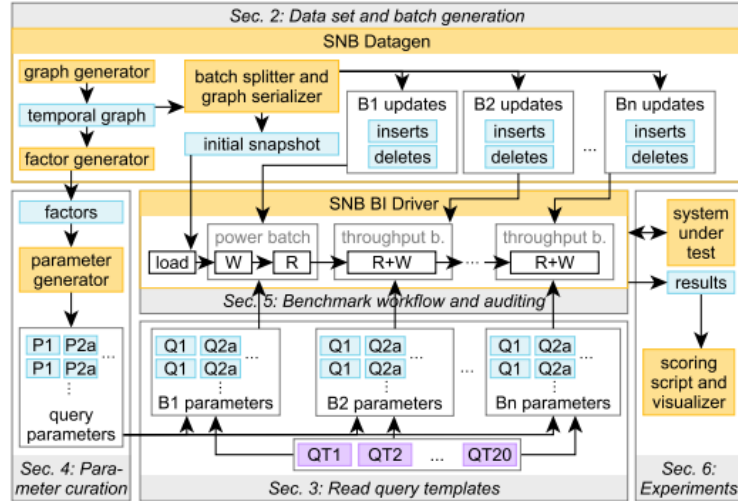
2.4. Teaching Examples from Published Papers

While reading the literature, I found also useful to look not only technical content of the papers, but also how they explain benchmark design in visual way. In my opinion, good figures in benchmarking papers are not just decorative things. More like teaching tools they are. They can show scope, experimental flow, workload orchestration, and also how results should be interpreted, much faster than only text. Because of that, I also selected some figures from published papers, as examples of effective benchmark communication.

Figure 1 is, for me, a good example for showing how a benchmarking paper can define benchmark scope and workload structure more clearly. This kind of figure is useful I think, because very early it tells the reader what is inside the benchmark, and how the main parts are related with each other.



(a) Model system implementing an ETL process from a transactional to an analytical data system and the systems targeted by each LDBC benchmark.



(b) Main components of SNB BI, and their connection to the workflow executed by the benchmark driver on the system under test. Writes (W) apply update operations, reads (R) execute queries.

Figure 1: Teaching example: benchmark scope visualization. Source: [1].

Figure 2, in my opinion, is a strong example for showing workflow. It makes the benchmark pipeline easier to follow by separating the generation, execution, and evaluation stages. I think this is exactly the kind of structure that a benchmarking paper should show clearly to the reader.

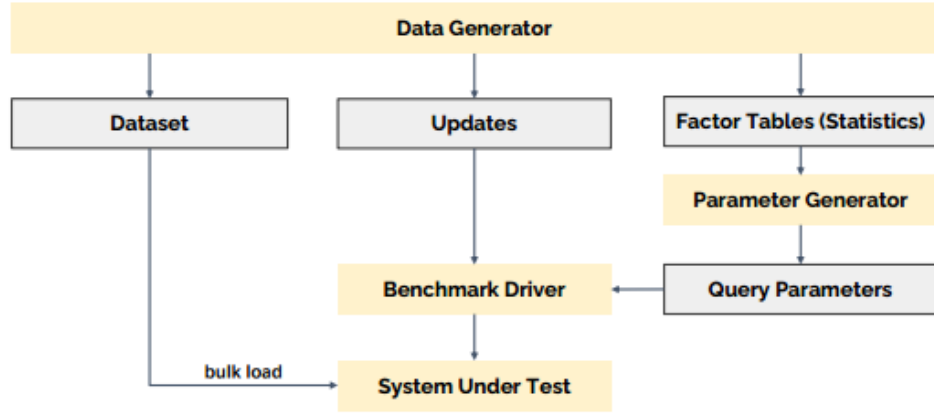


Figure 2: Teaching example: benchmark workflow visualization. Source: [2].

Figure 3 is useful because it shows a benchmark where workload is not static, but scheduled and orchestrated. I think this is especially helpful for mixed workload studies, because concurrency and timing matter there as much as query content.

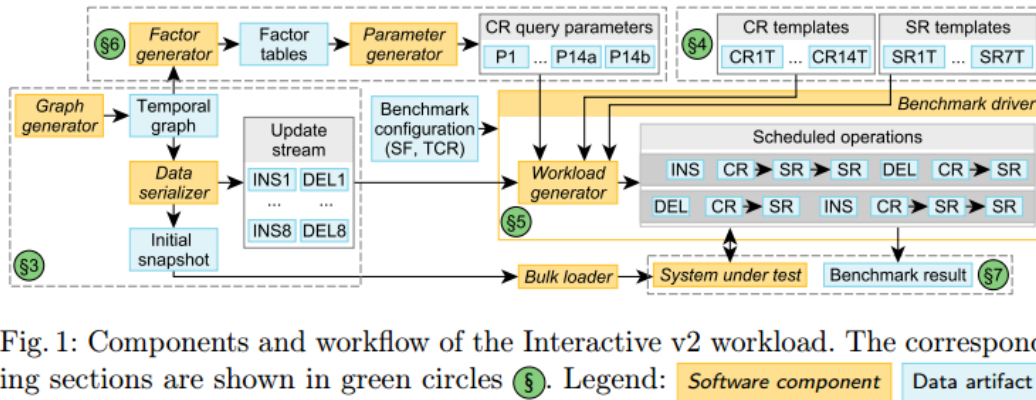


Fig. 1: Components and workflow of the Interactive v2 workload. The corresponding sections are shown in green circles \$\$. Legend: Software component Data artifact

Figure 3: Teaching example: workload scheduling and execution flow. Source: [3].

Figure 4 shows in my opinion, how a paper can present scalability behavior in a direct and readable way. A good benchmarking paper should not only report that performance changes with more resources; it should visualize that change in a way that immediately reveals the trend.

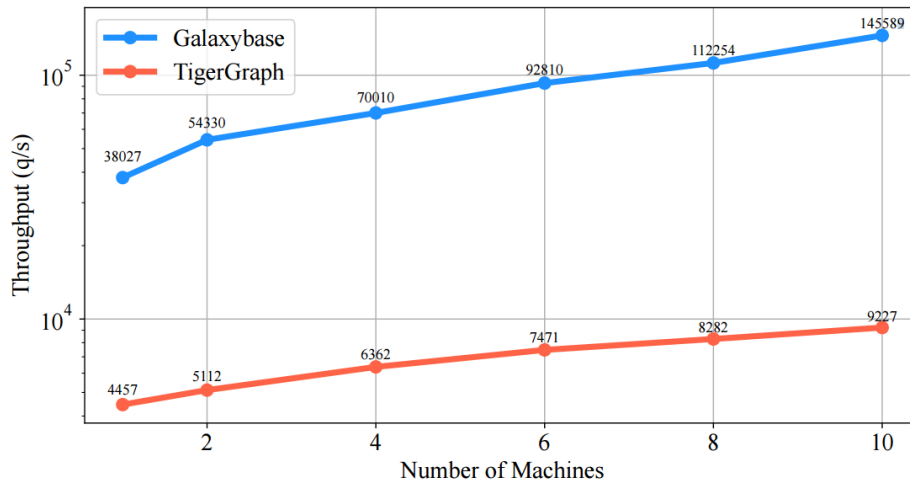


Figure 11: Scalability throughput for 1 to 10 machines

Figure 4: Teaching example: scalability-oriented result presentation. Source: [4].

Figure 5 is another strong teaching example because it presents scale-up and scale-out results in a way that encourages comparison across system configurations. I think this kind of figure helps turn raw performance measurements into a more interpretable systems story.

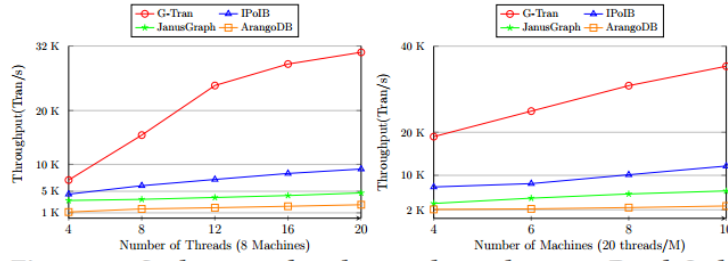


Figure 11: Scale-up and scale-out throughput on Read-Only workload over the LDBC-L dataset

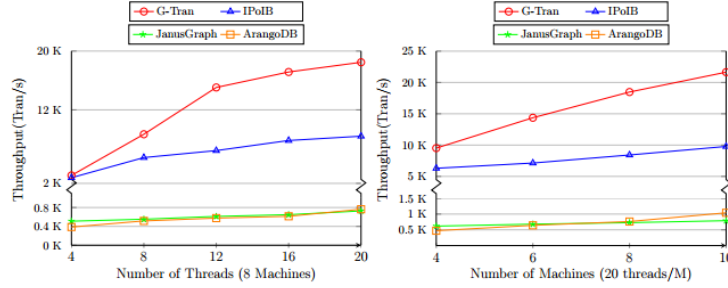


Figure 12: Scale-up and scale-out throughput on Read-Intensive workload over the LDBC-L dataset

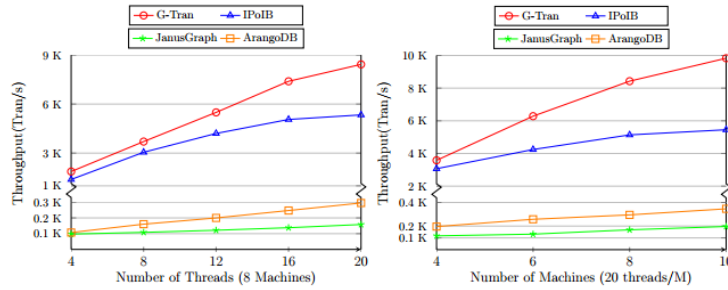


Figure 13: Scale-up and scale-out throughput on Write-Intensive workload over the LDBC-L dataset

Figure 5: Teaching example: scale-up and scale-out result presentation. Source: [5].

2.5. My Main Takeaways for Writing the Planned Paper

After reading these papers, the main idea staying with me was this: a benchmarking paper should be written like a structured scientific argument. In my opinion, first the paper should define a narrower but meaningful scope, then it should explain why benchmark design is built in that way, after that fair baselines should be given, then to mixed or more stressed scenarios it should move, and at the end the observed behaviors should be explained from system side. When there is this kind of order, I think, both easier the paper is to read and more convincing the results become.

I also think a good benchmarking paper should always make a balance between ambition and realism. A benchmark becomes more valuable when it touches an important problem, but more credible it becomes when assumptions, compromises because of portability, and limitations are

written clearly. In the examples that I read, the best papers were not the ones claiming they cover everything. More strong looked the papers which choose one specific problem, measure it carefully, and explain the results in a more honest way.

For my own planned paper, this means something like this: my benchmark design should be narrow enough so that it can be implemented carefully, but also broad enough to show meaningful trade-offs between systems. So in the end, things like structure, experiment design, and visual presentation, all of them should serve to this goal.

1. Part 3: Outline of the Planned Paper

Planned Paper Title

Benchmarking open source Distributed Property-Graph Databases with Portable Mixed Workloads under Elastic Scale-Out

1. Introduction

- In this section, I will explain why graph data management is becoming more important in areas like social networks, fraud detection, recommendation systems, and knowledge graphs.
- I will also explain why, in my opinion, it is not enough to evaluate graph systems only with OLTP-only or OLAP-only workloads.
- Here I will introduce the main motivation of my paper. I will say that literature already has strong benchmark-definition papers and also strong system papers, but still there is a place for a portable, open, multi-system benchmark protocol for mixed graph workloads under one controlled elasticity event.
- I will clearly state my main research question here. Basically, I will ask how open source distributed property-graph databases make balance between throughput, latency, and interference under portable mixed workloads, and how they behave when one controlled scale-out event happens.
- I will also make clear that this paper is a benchmarking paper, not a new system design paper.
- In the end of this part, I will mention that full scope of the study includes five systems, but if deployment side creates problems, I may start with smaller subset first.

2. Related Work

- In this section, I will explain the papers that are related with my topic, but not like one long summary list. I think that kind of writing becomes boring and also not very useful. Because of that, I will divide the literature into some more meaningful groups.
- First, I will talk about benchmark-definition papers.
 - Here I will discuss LDBC SNB BI as an important public benchmark source for analytical graph workloads, so more on the OLAP side.
 - I will also discuss LDBC SNB Interactive v2 as a public benchmark source for transactional graph workloads, so more on the OLTP side, and also for workload scheduling and concurrency side.
 - Then I will explain LDBC FinBench as another public benchmark source, especially useful because it brings graph transaction patterns from financial domain, not only from social-network side.
- After that, I will move to system papers.
 - In this part, I will discuss Galaxybase, G-Tran, ByteGraph, and BACH.
- Then I will give short background for the systems that I plan to benchmark.
 - Here I will briefly explain NebulaGraph, JanusGraph, HugeGraph, Dgraph, and ArangoDB.
 - I will mention their general design ideas, strong and weak sides and also portability or deployment problems that may matter for benchmarking.
- At the end of this section, I will connect all these groups together and explain what kind of gap I think still exists in the literature.

3. Systems Under Test

- In this section, I will present the final candidate systems:
 - NebulaGraph
 - JanusGraph
 - HugeGraph
 - Dgraph
 - ArangoDB 3.11.x
- I will explain why I selected these systems.
 - they are open source,
 - they support distributed deployment or distributed-ready setup,
 - they support graph data model,
 - they have practical relevance,

- and they look suitable for comparative benchmark study.
- I will also mention why ArangoDB is limited to 3.11.x line, because of later source-license complications.
- Then I will summarize each system in terms of query model, deployment style, and expected portability problems.
- If necessary, I may also add one feature-support matrix, so that portability and fairness limits can be seen more clearly.

4. Benchmark Design

- In this section, I will explain that my benchmark is not a completely new workload, and also not a claim of full reproduction of existing benchmark suites.
- I will define it more as a portable mixed workload protocol.
- Here I will explain the main design goals of the benchmark:
 - portability across different systems,
 - fairness across different query and storage models,
 - reproducibility,
 - realistic mixed workload behavior,
 - and one explicit controlled elasticity event.
- I will also define workload structure.
 - OLTP-style operations will be inspired mainly by LDBC SNB Interactive v2,
 - OLAP-style queries will be inspired mainly by LDBC SNB BI,
 - and LDBC FinBench will be used as one more public source for transaction-oriented graph patterns outside social-network domain.
- I will make clear that the benchmark uses only a small portable subset of operations.
- I will also explain why this kind of portable subset is needed, because systems do not support same query languages, same optimizers or same feature sets.

5. Public Benchmark Inputs and Scale Factors

- In this section, I will explain that I use three public benchmark input families and their public datasets:
 - LDBC SNB BI
 - LDBC SNB Interactive v2
 - LDBC FinBench
- I will explain what role each benchmark family has in my study.

- I will also discuss selected scale factors or dataset sizes in a way that stays realistic for multi-system deployment and repeated experiments.
- If I reduce dataset size, I will justify this with things like deployment cost, fairness, and reproducibility.
- I will also clarify that even if I use only portable subset of the official suites, these public inputs still remain the basis of workload design.

6. Metrics

- In this section, I will define the main evaluation metrics:
 - throughput,
 - average latency,
 - tail latency, if feasible,
 - mixed workload interference,
 - scale-out recovery / stabilization behavior,
 - correctness / result validity.
- I will explain how I interpret mixed workload interference.
- I will also define what I will count as successful stabilization after the scale-out event.
- If I can collect them in a consistent way, I may also include timeout rate, failed-query rate, and some resource-usage indicators.

7. Fairness and Portability Rules

- In this section, I will explain that systems will be compared only on a shared executable subset.
- If one query or feature is not supported, I will document this openly instead of hiding it.
- I will also explain the tuning policy.
 - each system can receive reasonable configuration support,
 - tuning should stay documented and balanced,
 - no system should get special unfair advantage.
- Here I will also describe rules for indexing, warm-up, retries, timeout handling, and repeated runs.
- If there are translation compromises between query languages, I will record and discuss them clearly.

8. Experimental Setup

- In this section, I will explain the hardware and software environment.
- I will report deployment assumptions for each system.
- I will also describe data-loading workflow, cluster setup assumptions, and runtime conditions.
- I will explain repetition policy and how I deal with result variability.
- I may also mention that experiment plan can start with smaller system subset before extending to all candidate systems.

9. Scenario Design

- In this section, I will define a small number of scenarios, because I want the study to stay realistic and focused.
- I am planning scenario groups like these:
 - S1: transactional-only baseline,
 - S2: analytical-only baseline,
 - S3: mixed workload with lower update intensity,
 - S4: mixed workload with higher update intensity,
 - S5: one controlled scale-out event during live mixed workload.
- I will explain why baseline scenarios are needed before mixed workload results can be interpreted.
- I will also explain why I include only one explicit scale-out event. Main reason is to keep the study narrow, interpretable, and feasible.

10. Results and Discussion

- In this section, I will present results in a more disciplined order.
 - first isolated transactional behavior,
 - then isolated analytical behavior,
 - then mixed workload behavior,
 - then scale-out behavior,
 - and finally cross-system trade-off discussion.
- I will try to use this section not only for listing graphs, but for answering concrete questions.
- Whenever possible, I will interpret system differences by looking at workload type, storage design, query model, or architecture side.
- I will also discuss where systems look strong, where interference becomes visible under mixed workloads, and how recovery looks after scale-out.

11. Threats to Validity

- In this section, I will openly discuss the main threats to validity.
- I will mention limits caused by query-language differences and translation compromises.
- I will also mention incomplete feature equivalence across systems.
- Dataset-size limits and scale-factor constraints will also be discussed here.
- I will also explain the limit of evaluating only one controlled scale-out event, instead of many different elasticity patterns.
- Finally, I will state clearly how these issues affect interpretation of the results.

12. Conclusion

- In the conclusion, I will summarize the main benchmark findings.
- I will reconnect these findings to the original research question.
- I will also highlight why having a portable, open, mixed workload benchmark view is valuable for distributed property-graph systems.
- At the end, I may briefly mention future extensions, like larger-scale execution, additional systems, or richer workload families.

References

- [1] Gábor Szárnyas, Jack Waudby, Xuan Zhou, Mingxi Wu, Andrey Gubichev, and Peter Boncz. The LDBC Social Network Benchmark: Business Intelligence Workload. *Proceedings of the VLDB Endowment*, 16(4):877–890, 2022. doi:[10.14778/3574245.3574270](https://doi.org/10.14778/3574245.3574270).
- [2] Shipeng Qi, Guodong Li, Zhaoyang Wang, Chao Li, Xianrui Qin, Xiang Hu, and Lei Zou. The LDBC Financial Benchmark: Transaction Workload. *Proceedings of the VLDB Endowment*, 18(9):3007–3020, 2025. doi:[10.14778/3746405.3746424](https://doi.org/10.14778/3746405.3746424).
- [3] David Püroja, Jack Waudby, Peter Boncz, and Gábor Szárnyas. The LDBC Social Network Benchmark Interactive Workload v2: A Transactional Graph Query Benchmark with Deep Delete Operations. *arXiv preprint arXiv:2307.04820*, 2023. doi:[10.48550/arXiv.2307.04820](https://doi.org/10.48550/arXiv.2307.04820).
- [4] Bing Tong, Ting Sun, Yansong Lv, Hong Gao, Jizhong Sun, Hongbo Zhao, Xiaowei Shen, Xiaogang Wang, Chunfeng Yuan, Shaohua Wang, and Weijian Yu. Galaxybase: A High Performance Native Distributed Graph Database for HTAP. *Proceedings of the VLDB Endowment*, 17(12):3893–3905, 2024. doi:[10.14778/3685800.3685814](https://doi.org/10.14778/3685800.3685814).

- [5] Hongzhi Chen, Yutong Lu, Qiang Cao, Xiaodong Lin, Xiaochun Yang, Gao Cong, and Aoying Zhou. G-Tran: A High Performance Distributed Graph Database with a Decentralized Architecture. *Proceedings of the VLDB Endowment*, 15(11):2545–2558, 2022. doi:[10.14778/3551793.3551813](https://doi.org/10.14778/3551793.3551813).
- [6] Changji Li, Zhenjia Li, Yuhan Sun, Yexuan Zhou, Yizhou Liu, Zhenhe Chen, Mingxing Zhang, Jian Tan, Minghui Zhang, Jianjun Chen, Zhenyu Li, Jianhua Liu, Xuemin Lin, and Wenfei Fan. ByteGraph: A High-Performance Distributed Graph Database in ByteDance. *Proceedings of the VLDB Endowment*, 15(12):3306–3318, 2022. doi:[10.14778/3554821.3554824](https://doi.org/10.14778/3554821.3554824).
- [7] Jianfeng Huang, Yihao Cao, Shubing Ren, Baohua Wu, and Dongjing Miao. BACH: Bridging Adjacency List and CSR Format Using LSM-Trees for HGTAP Workloads. *Proceedings of the VLDB Endowment*, 18(5):1509–1521, 2025. doi:[10.14778/3718057.3718076](https://doi.org/10.14778/3718057.3718076).
- [8] Min Wu, Xinglu Yi, Hui Yu, Yu Liu, and Yujue Wang. Nebula Graph: An Open Source Distributed Graph Database. *arXiv preprint arXiv:2206.07278*, 2022. doi:[10.48550/arXiv.2206.07278](https://doi.org/10.48550/arXiv.2206.07278).
- [9] The JanusGraph Authors. JanusGraph: Distributed Graph Database. GitHub repository, 2024. Accessed 4 Apr. 2026. URL: <https://github.com/JanusGraph/janusgraph>.
- [10] Apache Software Foundation. Apache HugeGraph. GitHub repository, 2024. Accessed 4 Apr. 2026. URL: <https://github.com/apache/hugegraph>.
- [11] Dgraph Labs. Dgraph: Native GraphQL Database with Graph Backend. GitHub repository, 2024. Accessed 4 Apr. 2026. URL: <https://github.com/dgraph-io/dgraph>.
- [12] ArangoDB GmbH. ArangoDB: The Multi-Model Database. GitHub repository, 2024. Accessed 4 Apr. 2026. URL: <https://github.com/arangodb/arangodb>.